



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Revaldo Fiqqih Septiansyah

NPM : 2428240110

Kelas : SI5C

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Revaldo Fiqqih Septiansyah
NPM	2428240110
Kelas	SI5C
Pertemuan Ke-	3-4
Topik / Judul Praktikum	Membangun RESTful API CRUD
Tanggal Pelaksanaan	21-22
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1.
mampu membuat backend RESTful API menggunakan Express.js.

2.

3.

II. Dasar Teori

Node.js dan Express.js

Node.js adalah runtime JavaScript berbasis mesin V8 milik Google Chrome yang memungkinkan JavaScript dijalankan di sisi server, dan menjadi fondasi bagi framework Express.js. Express.js adalah framework web minimalis untuk Node.js yang mempermudah pembuatan server dan API melalui routing sederhana dan middleware, tanpa banyak kode boilerplate (Rachmat, 2025).

RESTful API

REST (*Representational State Transfer*) adalah gaya arsitektur API berbasis resource yang diakses melalui HTTP. Prinsipnya: setiap data direpresentasikan sebagai URL (mis.

/mahasiswa), setiap request bersifat *stateless* (independen), operasi CRUD dipetakan ke HTTP method — GET (read), POST (create), PUT (update), DELETE (remove) — dan format pertukaran datanya berupa JSON (Fielding, 2000).

Anatomi Request dan Response

Pada Express.js, data dari client diperoleh melalui `req.params` (bagian dinamis URL), `req.query` (query string), dan `req.body` (body request, hanya terbaca setelah middleware `express.json()` didaftarkan). Server merespons melalui `res.json()` (mengirim data JSON), `res.status()` (kode status HTTP seperti 200, 201, 404), dan `res.send()` (Rachmat, 2025).

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	24.4.0
2	Visual Studio Code	1.139.1
3	Postman	9.26.8
4	Github	Platform web (github.com) — repository hosting
5	Vercel	Platform web (vercel.com) — deployment serverless

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1.

Persiapan environment dan inisialisasi project. Pastikan Node.js sudah terinstall melalui NVM dengan perintah `node -v`, lalu buat folder `tugas1-restful-hotel`, jalankan `npm init -y` untuk membuat `package.json`, dan instal Express.js dengan `npm install express`

2.

Membuat server Express.js. Buat file `app.js` sebagai entry point, tulis kode untuk mengimpor express, membuat instance app, serta menjalankan server pada PORT 3000 (atau `process.env.PORT`) menggunakan `app.listen()`, dan pastikan `main` di `package.json` mengarah ke `app.js`.

3.

Konfigurasi middleware dan data sementara. Mendaftarkan `app.use(express.json())` agar server dapat membaca `req.body`, kemudian mendeklarasikan array `rooms` berisi objek `{ id, nomorKamar, tipe, hargaPerMalam, kapasitas, tersedia }` sebagai sumber data sementara (in-memory), beserta variabel `nextId` untuk id otomatis.

4.

Membuat routing GET. Menambahkan route `GET /rooms` untuk menampilkan seluruh data (mendukung filter `req.query`, mis. `?tipe=deluxe`) dan `GET /rooms/:id` untuk menampilkan satu data berdasarkan `req.params`, dengan respons JSON 404 dan format `{ status, message, data: null }` jika data tidak ditemukan.

5.

Membuat routing POST. Menambahkan route `POST /rooms` untuk menambah data baru ke dalam array dari `req.body` dengan id dibuat otomatis oleh server, memvalidasi field wajib (`nomorKamar`, `tipe`, `hargaPerMalam`, `kapasitas`) — kirim 400 jika kosong — lalu mengirim respons dengan kode status 201.

6.

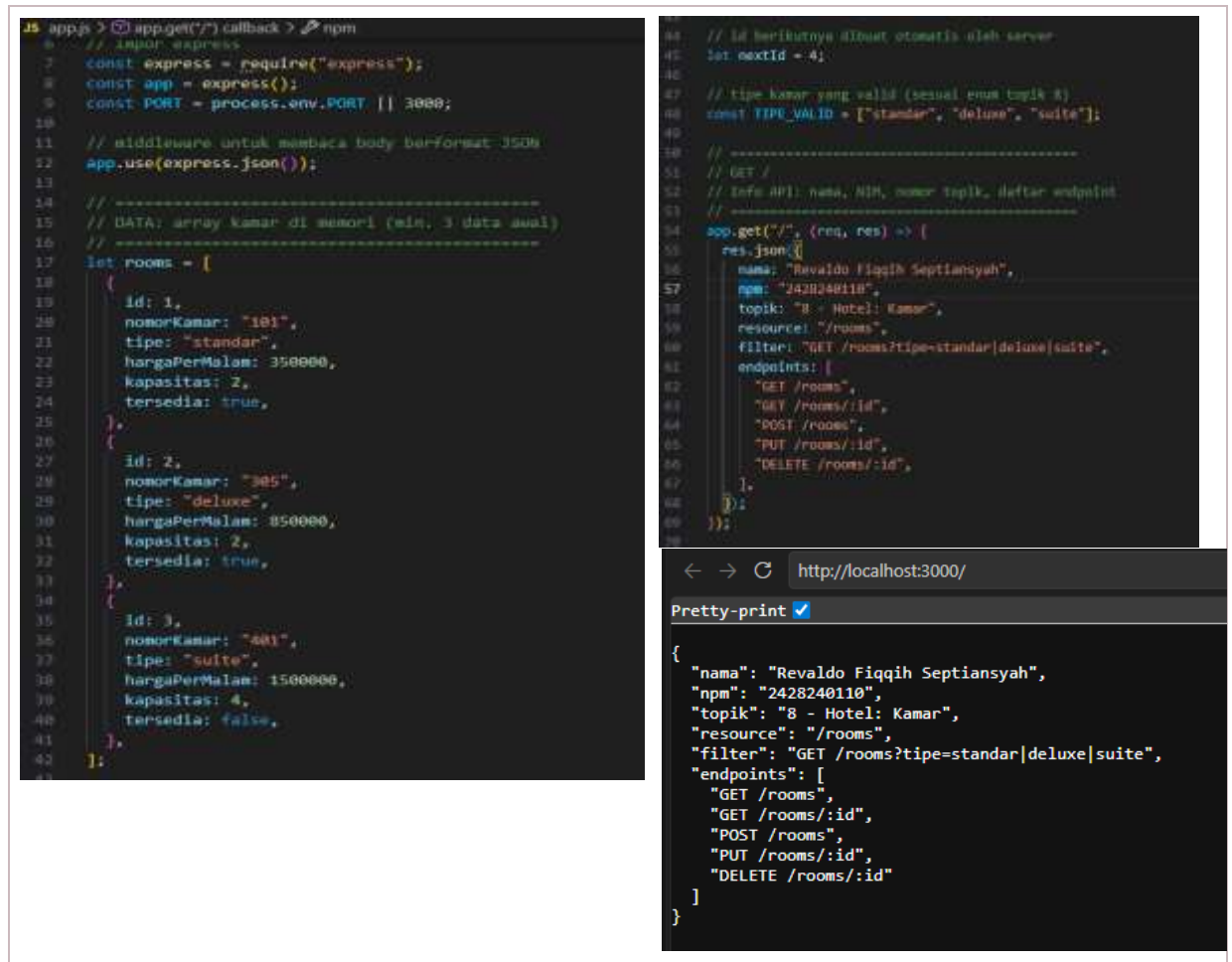
Membuat routing PUT dan DELETE. Menambahkan route `PUT /rooms/:id` untuk memperbarui seluruh data berdasarkan id (200 OK, 400 jika field wajib kosong, atau 404 jika tidak ada) dan `DELETE /rooms/:id` untuk menghapus data dengan `splice()` serta mengembalikan status 200 OK dengan `data: null` dan pesan yang menyebut id yang dihapus. Tambahkan juga middleware `catch-all 404` di akhir file untuk endpoint yang tidak terdaftar.

7.

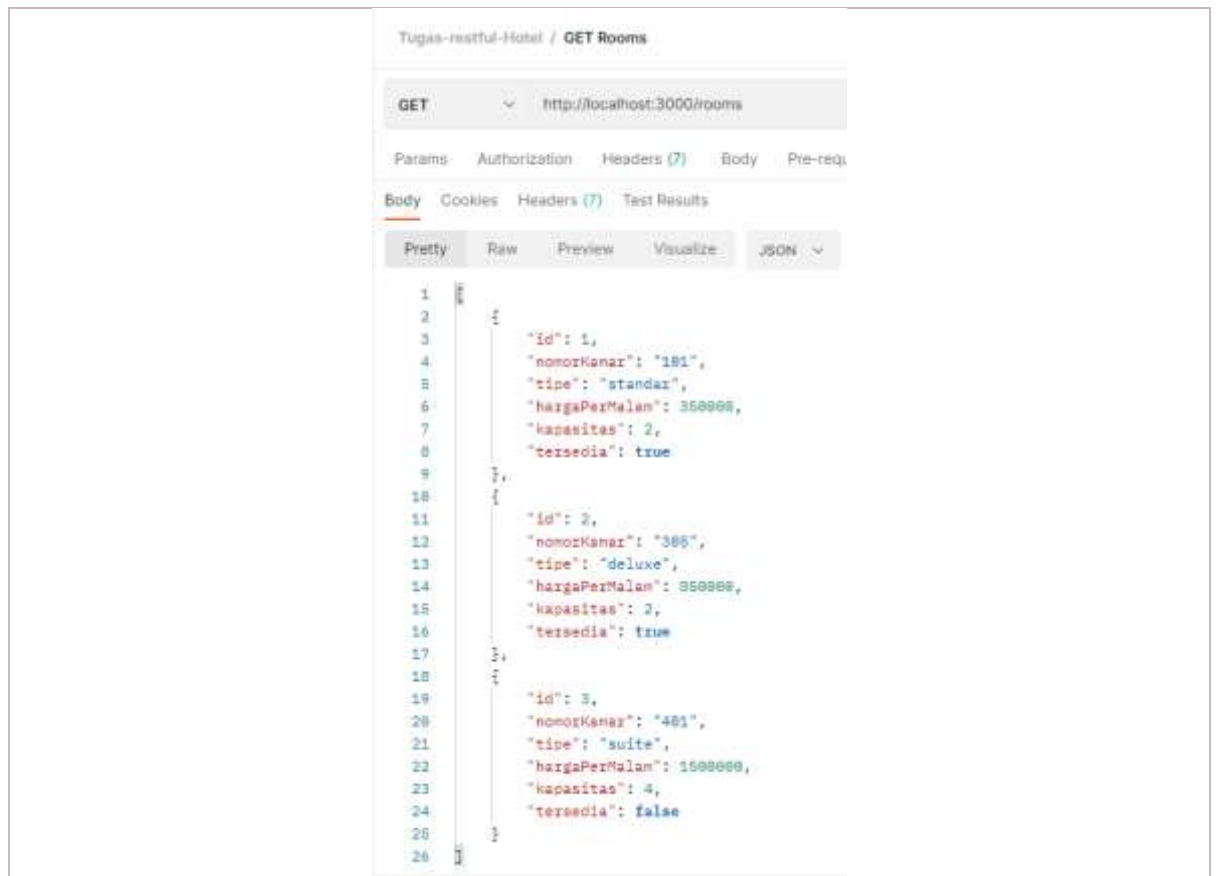
Menguji setiap endpoint. Menjalankan server dengan node app.js, lalu menguji seluruh endpoint (GET, POST, PUT, DELETE) menggunakan Thunder Client/Postman dan mencatat hasilnya dalam bentuk tangkapan layar untuk bagian Hasil dan Pembahasan.

V. Hasil dan Pembahasan

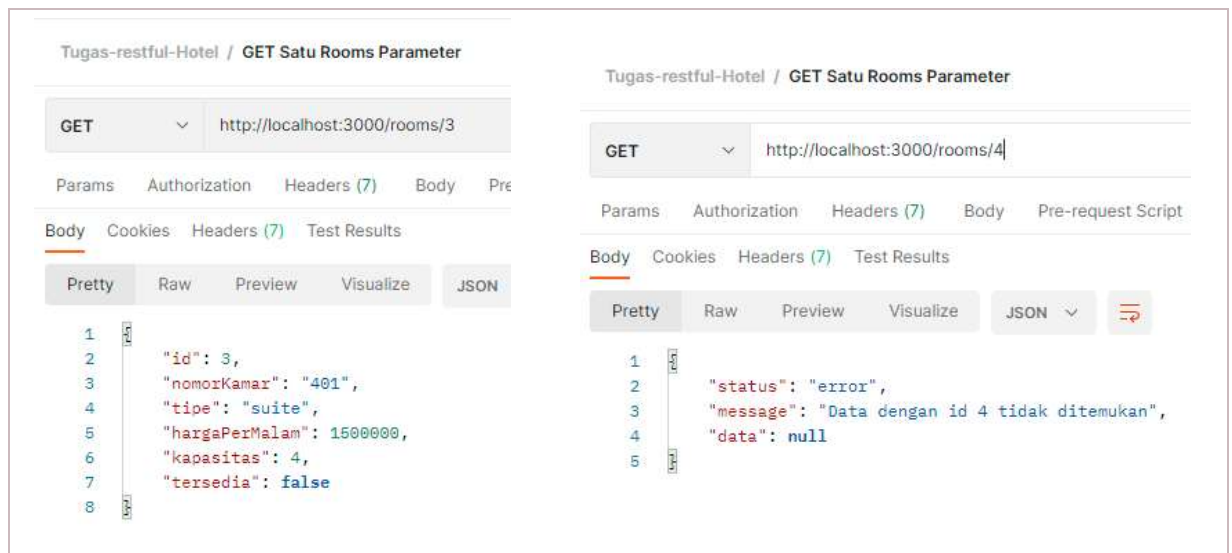
Sertakan tangkapan layar (screenshot) hasil praktikum, disertai penjelasan/pembahasan pada setiap hasil.



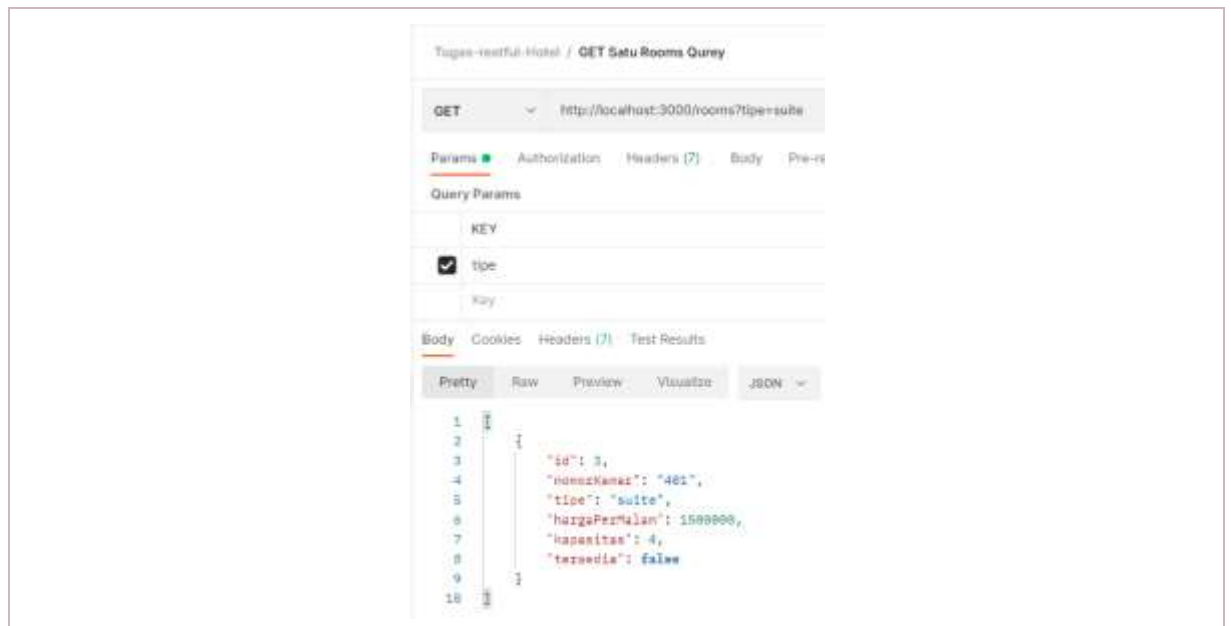
Gambar 1.Setup Persiapan



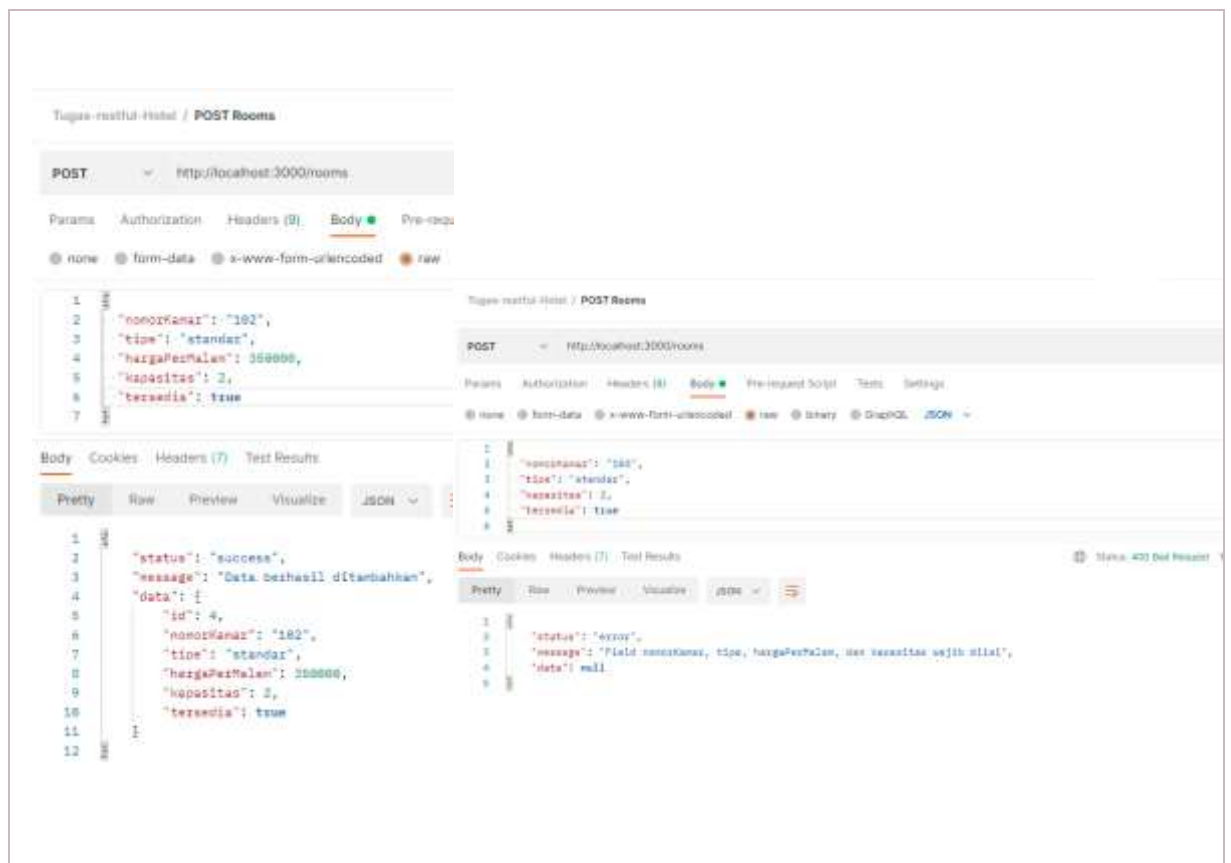
Gambar 2. GET Rooms



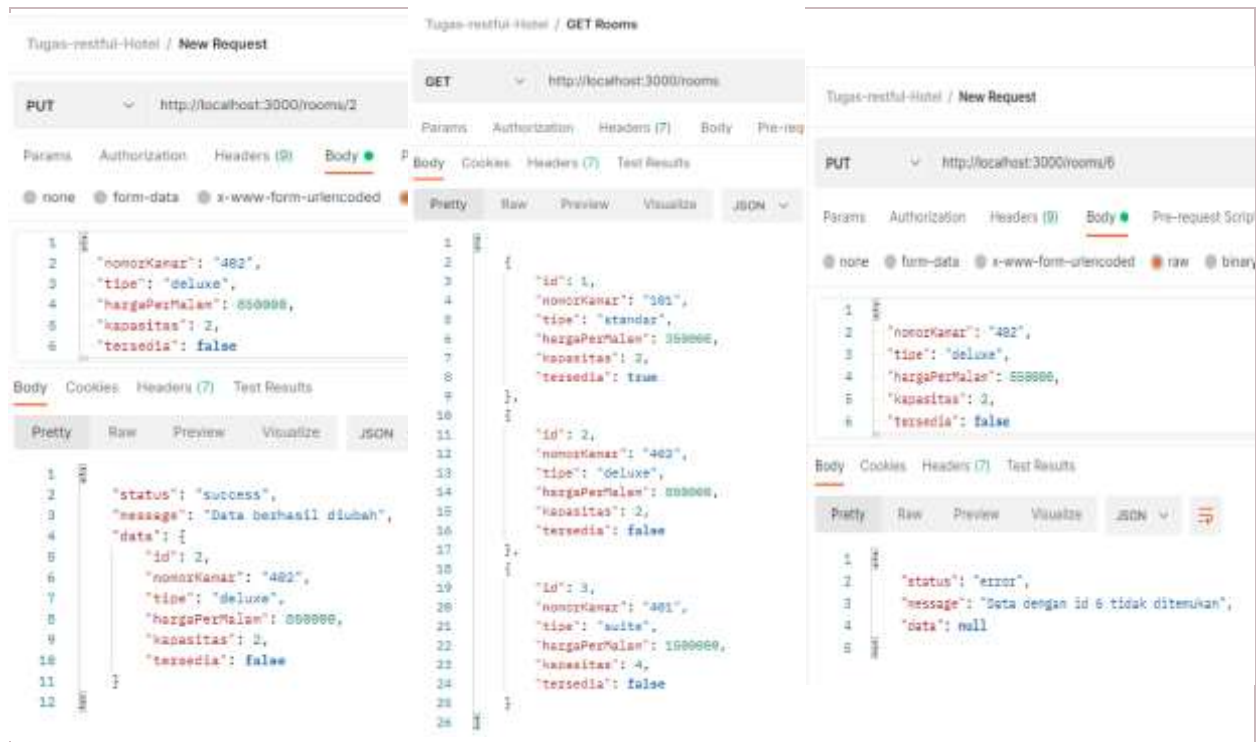
Gambar 3. GET Rooms Parameter



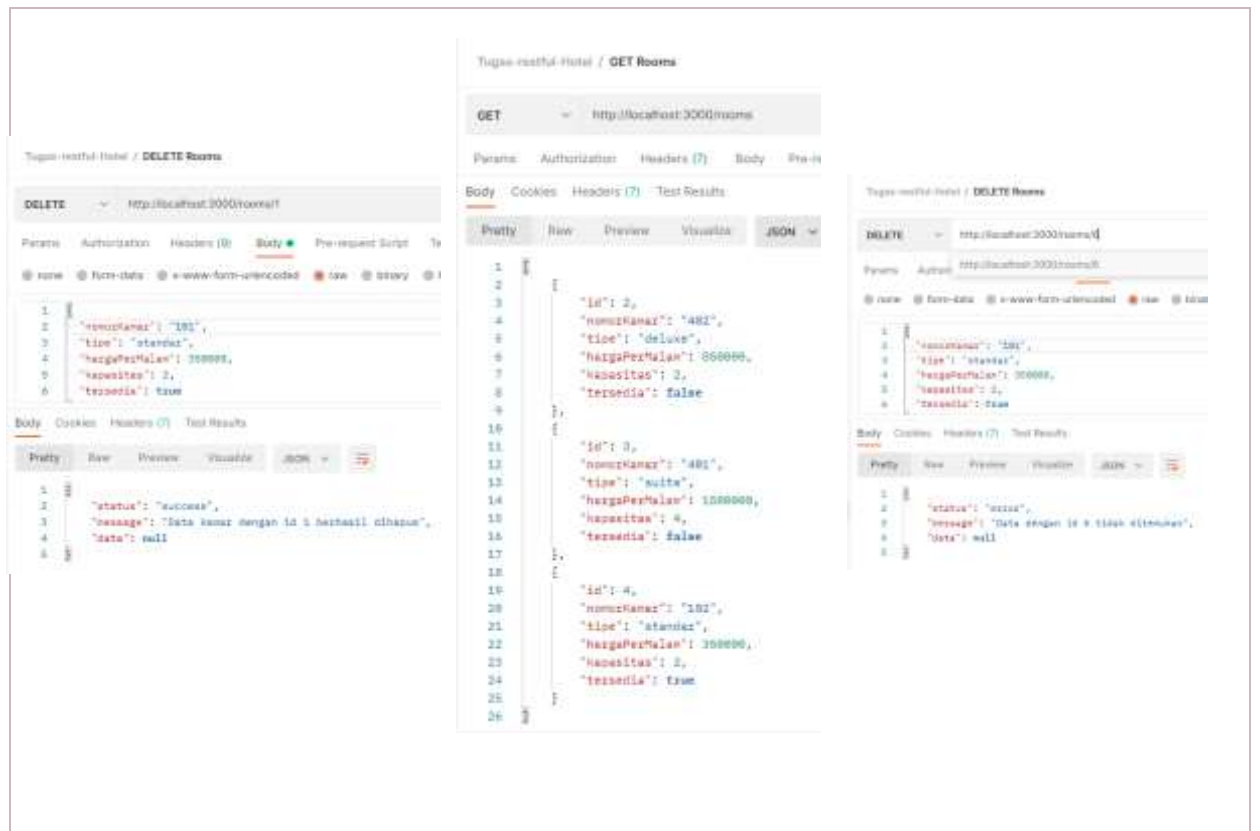
Gambar 4. GET Satu Rooms Qurey



Gambar 5. POST Rooms



Gambar 6. PUT Rooms.



Gambar 7. DELETE Mahasiswa

*Pembahasan:***Gambar 1. Setup Persiapan**

Melakukan setup server dengan Node.js dan Express.js, menginisialisasi project dengan npm init -y, menginstal express dan nodemon, mengatur app.js sebagai entry point yang berjalan di `http://localhost:3000`, mendaftarkan middleware `express.json()`, serta membuat array `rooms` berisi 3 data awal kamar (standar, deluxe, suite) beserta variabel `nextId` untuk id otomatis.

Gambar 2. GET Mahasiswa

Menampilkan seluruh data kamar dengan metode GET tanpa parameter. Server mengembalikan array berisi 3 objek kamar dengan status 200 OK. Method GET dipilih karena operasi ini hanya membaca data tanpa mengubah array.

Gambar 3. GET Mahasiswa Parameter

Menampilkan satu data kamar berdasarkan route parameter id menggunakan `req.params.id`. Request GET `/rooms/1` mengembalikan objek kamar dengan id 1 dan status 200. Method GET digunakan karena kembali hanya membaca satu data. (Jika pada gambar juga ditampilkan GET `/rooms/4` yang belum ada datanya, server mengembalikan status 404 dengan format `{ status: "error", message: "Data dengan id 4 tidak ditemukan", data: null }` — hal ini membuktikan validasi data tidak ditemukan berjalan.)

Gambar 4. GET Satu Mahasiswa Query

Menampilkan data kamar dengan metode GET menggunakan query string `?tipe=deluxe` melalui `req.query.tipe`. Server memfilter array dengan `rooms.filter()` sehingga hanya kamar bertipe deluxe yang dikembalikan dalam bentuk array dengan status 200. Query string dipilih (bukan route parameter) karena filter bersifat opsional dan tidak menunjuk id tertentu.

Gambar 5. POST Mahasiswa

Menambahkan data kamar baru ke dalam array. Pada request pertama dengan body lengkap (nomorKamar, tipe, hargaPerMalam, kapasitas, tersedia), server memvalidasi field, membuat id otomatis dengan `nextId++`, menambahkan data dengan `push()`, dan mengembalikan status 201 Created beserta objek data baru. Status 201 dipilih karena resource baru berhasil dibuat. Pada request kedua, field wajib sengaja dikosongkan (misalnya tanpa `hargaPerMalam`), sehingga server menolak dengan status 400 Bad Request sesuai ketentuan validasi.

Gambar 6. PUT Mahasiswa

Mengubah seluruh field data kamar dengan metode PUT (penggantian penuh). Pada PUT `/rooms/1` dengan body lengkap, seluruh field kamar id 1 diganti dan server mengembalikan status 200 OK bersama data terbaru. Method PUT dipilih karena mewakili operasi update/replace pada resource yang sudah ada. Pada PUT `/rooms/6`, karena id 6 tidak ditemukan di array, server mengembalikan status 404 Not Found.

Gambar 7. DELETE Mahasiswa

Menghapus data kamar dengan metode DELETE berdasarkan `req.params.id` menggunakan `splice()`. Pada DELETE `/rooms/1`, data berhasil dihapus dan server mengembalikan status 200 OK dengan data: null serta pesan "Data kamar dengan id 1 berhasil dihapus". Pada DELETE `/rooms/6`, karena data tidak ditemukan, server mengembalikan status 404 Not Found.

VI. Kendala dan Solusi

Opsional - isi apabila terdapat kendala teknis selama praktikum.

No.	Kendala	Solusi
-----	---------	--------

1	req.body undefined saat POST/PUT	menambahkan <code>app.use(express.json())</code> sebelum semua route dan memastikan request dikirim sebagai raw JSON.
2		

VII. Kesimpulan

Membuat *backend* RESTful API menggunakan Express.js merupakan proses membangun sistem yang mengatur lalu lintas data antara aplikasi dan database secara efisien. Melalui framework ini, empat perintah utama dalam pengelolaan data berhasil diimplementasikan, yaitu **GET** untuk mengambil dan menampilkan data, **POST** untuk menambahkan data baru dari pengguna, **PUT** untuk mengubah data lama yang sudah ada, serta **DELETE** untuk menghapus data dari sistem. Penguasaan Express.js beserta keempat metode HTTP tersebut menunjukkan kemampuan dalam menciptakan sistem *backend* utuh yang siap dihubungkan ke berbagai jenis aplikasi, baik website maupun aplikasi *mobile*.

Daftar Pustaka

Tuliskan sumber rujukan yang digunakan (format APA 7th edition).

Field, R. (2022). *Modern Web API Design with REST*. Academic Press.

Hakim, L. (2024). *Pengembangan Backend Modern dengan Node.js dan Express*. Jakarta: Penerbit Informatika.

Syafrizal, M., & Kurniawan, A. (2023). Analisis Kinerja Arsitektur RESTful API pada Sistem Informasi Akademik. *Jurnal Teknologi Informasi dan Komputer*, 9(2), 145-152.

Lampiran

Tautan Repository GitHub:

<https://github.com/revaldofiqqihseptiansyah2428240110/tugas1-restful-2428240110.git>

Tautan Deployment (jika ada):

<https://restful-hotel-beta.vercel.app/>